

NAVIGATING UNCERTAINTY: AN APPROACH TO MITIGATE SAMPLING ERRORS IN PORTFOLIO OPTIMIZATION

Aditya Santosh ^[1], Arun Govind ^[2], Rajeev Surve ^[3], Shail Talati ^[4]

Department of Quantitative Finance, Rutgers Business School, Newark

ABSTRACT

Effective portfolio optimization is crucial for maximizing returns and minimizing risks in investment management. However, traditional optimization approaches often suffer from significant sampling errors due to their reliance on historical data for forecasting. This project delves into portfolio optimization and efficient frontier analysis using historical returns data from stocks. The project methodology involves sampling returns to evaluate sampling errors. By systematically assessing sampling errors, we aim to understand the discrepancies between the efficient frontier generated from sampled returns and the frontier derived from historical data. We explore how these errors influence the expected returns and volatility of the portfolios. Additionally, the study incorporates variance reduction methods on sampled returns to assess their impact on expected returns and volatility. This research provides insights into the impact of sampling errors and the efficacy of variance reduction methods. The findings have implications for investment decision-making, risk management, and the development of more robust portfolio optimization strategies in financial markets.

I. INTRODUCTION

Despite significant advancements in financial modeling, the enduring reliance on historical data for predicting asset behavior introduces critical vulnerabilities. Effective portfolio management is extremely important, demanding not only strategic acumen but also effective analytical tools that predict and optimize the behavior of financial assets under varying market conditions. Portfolio optimization is a critical discipline, central to the strategic management of investment portfolios. Traditional portfolio optimization models, like the Markowitz mean-variance framework, have offered foundational insights into risk-return trade-offs but are critically hampered by the inherent limitations posed by sampling errors. These errors, arising from the reliance on historical data to estimate future returns and covariances, can significantly distort the optimization process, leading to investment strategies that perform sub-optimally under real-world conditions.

This paper seeks to address these challenges by integrating advanced variance reduction techniques into the portfolio optimization process, specifically through the use of Monte Carlo simulations complemented by the antithetic variates and moment matching methods. This approach is inspired by similar strategies in derivative pricing, where Monte Carlo methods are crucial in managing the complexity and variability of financial derivatives. By adopting these advanced simulation techniques, the study aims to mitigate the impact of sampling errors on the estimation of future asset returns and covariances, thereby enhancing the accuracy and reliability of the portfolio optimization process. It details the creation of a simulated investment portfolio consisting of 20 diverse stocks. This portfolio serves as a testbed for creating the joint distribution of returns and applying variance reduction techniques to estimate these returns more

accurately. Specifically, the paper investigates the efficacy of the variance reduction techniques in Monte Carlo simulation in reducing the variance of the return estimates sampled from the joint distribution, thereby providing a more stable and predictable foundation for optimizing portfolio allocations.

The methodology involves constructing a joint normal distribution of stock returns, simulating various portfolio scenarios, and applying plain Monte Carlo simulation (MC), Monte Carlo simulation with antithetic variables (MC-Anti), and Monte Carlo simulation with moment matching (MC-MM). We attempt these approaches to sharpen the precision of return estimations by effectively reducing the variance associated with random sampling errors. The expected result is a set of portfolio optimization strategies that are both theoretically sound and empirically robust, offering significant improvements over traditional models in terms of both risk management and return optimization.

The findings are expected to benefit academic researchers and financial practitioners alike, providing them with advanced tools to craft more effective investment strategies that are resilient in the face of the unpredictable dynamics of global financial markets.

II. LITERATURE REVIEW

Sampling error in portfolio optimization primarily arises from the need to estimate future asset returns and covariances from historical data. The degree of error is contingent upon the length of the data period, the volatility of asset returns, and the dimensionality introduced by the number of assets.

^[1] as4385@scarletmail.rutgers.edu, ^[2] ag2360@scarletmail.rutgers.edu, ^[3] rs2257@scarletmail.rutgers.edu, ^[4] st1251@scarletmail.rutgers.edu

Jobson and Korkie (1980) and Michaud (1989) illustrate how increased asset numbers amplify estimation errors due to the "curse of dimensionality," leading to unreliable efficient frontiers. This issue arises because each additional asset not only contributes its own variance but also its covariance with every other asset. The total number of covariances increases quadratically with the number of assets, making the covariance matrix denser and more complex to estimate accurately. When the efficient frontier is based on inaccurate estimates of returns and covariances, the resulting portfolio may be suboptimal, exposing the investor to higher risk or lower returns than anticipated. This is particularly problematic for mean-variance optimization, which relies heavily on these parameters to determine the optimal asset weights.

Mean-variance optimization, as introduced by Markowitz (1952), is a foundational concept in portfolio management, emphasizing the trade-off between risk and return. Despite its widespread use, mean-variance optimization is highly sensitive to the accuracy of the input data—specifically, the estimates of expected returns and the covariance matrix of the asset returns. This sensitivity significantly impacts the reliability of the optimized portfolios, especially when faced with sampling errors.

Impact of Sampling Errors on Mean-Variance Optimization:

1. Britten-Jones (1999) illustrates how sampling errors can lead to extreme weights in the optimized portfolio. For instance, if the estimated return for a particular asset is erroneously high due to a sampling anomaly, mean-variance optimization may allocate a disproportionately large weight to this asset, underestimating its risk. Conversely, if the return is underestimated, the asset might be unjustly excluded or underweighted. These extreme weights arise because mean-variance optimization inherently leverages the data to find the portfolio with the highest expected return per unit of risk based on the quadratic programming of the covariance matrix against expected returns. When the input data (returns and covariances) are inaccurate, the optimization process magnifies these inaccuracies, leading to portfolios that are far from optimal.
2. The efficient frontier in the mean-variance context represents the set of portfolios that offer the highest expected return for a given level of risk. Sampling errors in the estimates of returns and covariances cause the calculated efficient frontier to deviate from the true efficient frontier. This deviation implies that the portfolios deemed optimal based on historical data may not perform as expected in the future because they are based on erroneous assessments of the risk-return trade-off.
3. The primary consequence of these errors is poor out-of-sample performance. Portfolios optimized on historical data often fail to deliver the expected returns or maintain the predicted risk levels when applied to new, unseen data sets. This issue is particularly prevalent in financial markets where historical performance is not always indicative of future results. The optimization based on flawed estimates leads to a selection of portfolios that are not truly risk-efficient, thereby increasing the likelihood of underperformance.

Mitigation Strategies

1. Shrinkage Estimators: Ledoit and Wolf (2004) propose shrinking the sample covariance matrix towards a structured estimator like the identity matrix, which stabilizes the inversion process and reduces the estimation error. Shrinkage techniques adjust the covariance matrix towards a more stable predictor (like the identity matrix), reducing the impact of estimation errors in the sample covariance. This approach helps in stabilizing the optimization process by introducing a bias that reduces variance in the estimates.
2. Resampling Techniques: Michaud (1998) introduces resampling techniques to create a more robust estimation of the efficient frontier by generating multiple portfolios from perturbed versions of the estimated parameters and averaging the results.
3. Robust Optimization: Fabozzi et al. (2007) discuss robust optimization methods that formulate the problem within a worst-case scenario framework to hedge against parameter uncertainty. Robust optimization frameworks incorporate the uncertainty in parameter estimates directly into the optimization process. By considering worst-case scenarios within a defined uncertainty set, these models seek to find portfolios that are optimal under a variety of different conditions, thus reducing the sensitivity to errors in any single estimated parameter.

Recent Developments

The integration of advanced computational techniques and the increased availability of data have significantly transformed the landscape of portfolio optimization. Modern approaches, particularly those incorporating machine learning (ML) methodologies, offer promising solutions to mitigate the effects of sampling errors in estimating asset returns and covariances. These developments are pivotal as they address the inherent limitations of traditional optimization methods under parameter uncertainty.

Research by DeMiguel et al. (2009) has empirically demonstrated the effectiveness of machine learning techniques in portfolio optimization. Their studies show that portfolios constructed using ML techniques often outperform traditional mean-variance optimized portfolios, especially in terms of out-of-sample risk-adjusted returns. These findings underscore the potential of machine learning to enhance the robustness of portfolio optimization processes against sampling errors. By employment of cross-validation and other regularization techniques, he achieved optimized portfolios under parameter uncertainty. Cross-validation evaluates the generalizability of a model by partitioning data into subsets, using some subsets to train the model and others to test it. It is useful in financial modeling to prevent overfitting. Whereas Regularization adds a penalty on the size of the coefficients in regression models, which can be used to handle multicollinearity, enhance prediction accuracy, and prevent overfitting.

III. THEORETICAL FOUNDATION

Variance Reduction Techniques

To enhance the accuracy and efficiency of the Monte Carlo simulation, variance reduction techniques are essential:

- **Antithetic Variates** - The principle behind the antithetic variates technique is to utilize the property of symmetry in normal distributions to reduce the variance of the estimator. By generating pairs of negatively correlated sample paths, the technique aims to cancel out some of the variability observed in the estimation process.
- Suppose we need $2n$ random variables which we generate using random uniform from 0 to 1.
- Normally, one would generate $2n$ random uniform numbers between 0 and 1.
- In the antithetic variates method, for each uniform random number X_i , a corresponding antithetic sample $1-X_i$ is also generated.
- Let us take two streams of uniform random numbers, $X_1, X_2, X_3, \dots, X_n$ and $1-X_1, 1-X_2, 1-X_3, \dots, 1-X_n$.
- We can write the combination of these generated random numbers as follows:

$$\bar{X} = \frac{X_1 + X_2}{2n}$$

where X_1 and X_2 are the sum of the two streams of random numbers we have utilized respectively.

- Variance Reduction: The variance is given by:

$$Var(\bar{X}) = \frac{1}{4} \left[Var\left(\frac{X_1}{n} + \frac{X_2}{n}\right) \right]$$

$$Var(\bar{X}) = \frac{1}{4} [Var(\bar{X}_1) + Var(\bar{X}_2) + cov(\bar{X}_1, \bar{X}_2)]$$

If we want to reduce the variance, we need to choose covariance such that it reduces the overall $Var(\bar{X})$.

- **Moment Matching** - It involves adjusting the simulated data so that certain statistical moments (typically the first and second moments - mean and variance) of the simulation match those specified by the theoretical distribution. This method corrects any bias in the simulations introduced by sampling errors.
- ❖ Let Z be a vector of simulated random variables thought to follow a normal distribution with expected mean μ and variance σ^2 , but which exhibit empirical mean $\bar{z}$ and empirical variance s^2 .

- ❖ Adjustment for Mean and Variance: The adjusted variable Z' is given by:

$$Z' = \sigma \frac{(Z - \bar{z})}{s} + \mu$$

This transformation adjusts the samples so that their mean and variance match the desired values μ and σ^2 .

- ❖ Effectiveness- By ensuring that the simulated data conform to the pre-specified moments, the fidelity of the simulation relative to the theoretical model is enhanced, thereby improving the accuracy.

Cholesky Decomposition for Correlated Random Variables

The Cholesky Decomposition method is used to transform independent standard normal variables into correlated variables consistent with a specified covariance matrix, C . This transformation is crucial when dealing with multiple underlying assets or advanced models that require a specific correlation structure between different variables. The matrix C is decomposed into an upper triangular matrix A such that $C = A^T A$.

This method ensures that the simulated variables exhibit the correct variance-covariance structure dictated by C , enhancing the accuracy of the simulation where asset returns are interdependent.

By incorporating these advanced methods into the Monte Carlo simulation for portfolio optimization, the methodology not only addresses the inherent limitations of traditional models but also leverages cutting-edge financial theory to provide more reliable and robust investment strategies.

IV. METHODOLOGY

The methodology employed in this study leverages simulation techniques and variance reduction methods to enhance portfolio optimization. The process is organized into several key stages: Data Preparation, Portfolio Optimization Setup, Monte Carlo Simulations with Variance Reduction, and Evaluation of the Models.

A. Data Preparation

The foundation of any robust financial analysis is high-quality data. This project starts with the collection and processing of monthly historical return data for a diverse portfolio of 20 stocks (Exxon Mobil (XOM), General Electric (GE), Chevron (CVX), Apple (AAPL), Procter & Gamble (PG), Southern Company (SO), Boeing (BA), Pfizer (PFE), Johnson & Johnson (JNJ), NextEra Energy (NEE), Ford Motor (F), Duke Energy (DUK), JPMorgan Chase (JPM), Walmart (WMT), Bank of America (BAC), Freeport-McMoRan (FCX), Amazon (AMZN), American Tower (AMT), NVIDIA (NVDA) and Alphabet (GOOGL)

from Wharton Research Data Services (WRDS).

The data includes daily returns over 10 years (from Jan'14 - Dec'23). Each stock's return data is cleansed to ensure consistency and accuracy, which involves handling missing values and normalizing the data across different scales.

B. Portfolio Optimization Setup

Portfolio optimization is performed using the mean-variance optimization framework. This approach involves determining the set of portfolio weights that minimize risk for a given level of expected return. Additionally, the efficient frontier is simulated, which represents the best possible returns for a given level of risk across various portfolios. This allows for the visualization of potential investment outcomes and helps in identifying the most efficient portfolios.

C. Monte Carlo Simulations and Variance Reduction Techniques

To estimate the performance of various portfolio weights under different market conditions, Monte Carlo simulations are employed. These simulations generate numerous possible scenarios for future returns based on the historical data, providing a probabilistic view of potential outcomes. To improve the accuracy of these simulations, variance reduction techniques are applied:

1. **Antithetic Variates:** This method involves generating pairs of negatively correlated variables to reduce the variance of the simulation estimates. By using both the original and antithetic variables, the simulation's accuracy is enhanced, as the opposing deviations from the mean tend to cancel out each other.
2. **Moment Matching:** Also known as the control variates method, this technique adjusts the simulated data to more closely match the statistical moments (such as mean and variance) of the theoretical distribution. This helps correct biases in the simulation results and reduces the estimation error.

We carry out two ways of sampling error analysis. First, the effect of sampling error as the sample size increases. The sample size is the historical data considered from the available 10 years of data. For eg. The return matrix is of size 20x120 and the sample size is 2 years (20x24). The final return matrix will include 20x24 returns from historical data and the rest of 20x96 will be randomly sampled from the joint normal distribution creating sampling errors. Second, the effect over the number of iterations of the Monte Carlo simulations.

D. Evaluation Metrics

The performance of the optimized portfolios is assessed using various error metrics, including Mean Squared Error (MSE) and Mean Absolute Error (MAE). These metrics provide a quantitative measure of the accuracy of the return predictions generated by the simulation models. By comparing these metrics across different models, the effectiveness of the variance reduction techniques can be evaluated, allowing for the refinement of the simulation processes.

E. Implementation and Software

The entire analysis is conducted using Python, leveraging its powerful libraries for numerical computation, data manipulation, and visualization. This includes using libraries like NumPy for handling large arrays of data, pandas for data processing, and matplotlib for creating charts and graphs that help visualize the results. The choice of Python and its libraries ensures an efficient workflow capable of handling the complex calculations required in this study.

V. ANALYSIS

Minimum Variance Portfolio

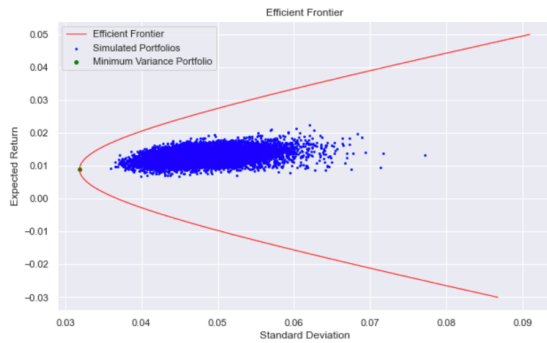
```
Expected Return(MVP): 0.00891281
Variance(MVP): 0.00101574
Weights(MVP): [ 0.20255166  0.06359329 -0.13817115 -0.0635115  0.24676046 -0.11179506
-0.0372397  0.05169321  0.08773159  0.12667283 -0.05688699  0.23469444
0.18882701  0.15092389 -0.08299858 -0.00745554  0.07766664  0.00788634
0.00138995  0.0576672 ]
```

The optimization process yields the weights of a Minimum Variance Portfolio (MVP), which is designed to minimize the risk (variance) for a given expected return. Here's an analysis of the results:

- ❖ **Expected Return (MVP):** The expected annual return of the MVP is approximately 0.89%. It's a relatively low return, suggesting either a conservative portfolio or a market environment with lower expected returns.
- ❖ **Variance (MVP):** The variance of the MVP is about 0.10%. Variance is a measure of the dispersion of returns. Since standard deviation is the square root of variance, the standard deviation (volatility) would be approximately $\sqrt{0.0010} \approx 3.16\%$. This indicates a relatively low level of risk associated with the MVP.
- ❖ **Weights (MVP):** The weights array represents the proportion of the total portfolio allocated to each stock. From the output, it is observed that some weights are negative, which indicates a short position in that stock. This portfolio is quite aggressive in terms of diversification, as it includes long positions and significant short positions, suggesting the use of leverage and a complex strategy that goes beyond simple buy-and-hold. This kind of strategy would generally be employed by sophisticated investors or hedge funds.

The presence of both long and short positions can also be part of a market-neutral strategy, aiming to profit from the relative performance of the stocks rather than the market's overall direction.

Risk-Return Trade-off Analysis for Optimized Portfolio



- **Efficient Frontier:** The red curve represents the efficient frontier derived from the combination of the 20 stocks in our portfolio.
- **Simulated Portfolios:** The blue dots are the expected returns of the inefficient portfolios in mean- σ space. Each dot corresponds to a portfolio with a specific combination of weights assigned to the 20 stocks. These simulations provide a visual representation of the risk-return profile for a vast number of potential portfolios. The scatter of blue dots shows how varying the weights of the stocks in the portfolio affects the expected return and risk.
- **Minimum Variance Portfolio (MVP):** The point marked in green on the efficient frontier represents the MVP. The MVP indicates the portfolio with the lowest possible risk among all combinations of the 20 stocks.

Error Metrics (Weights) – Exhibit A

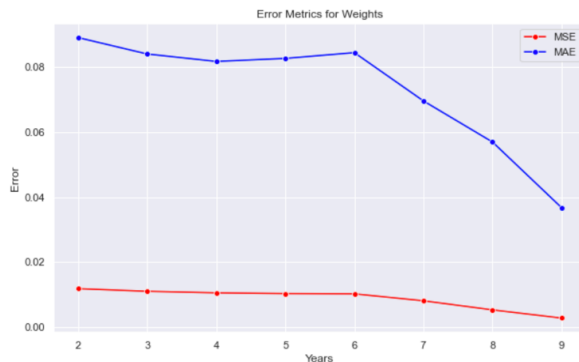


Exhibit A depicts the Mean Squared Error (MSE) and Mean Absolute Error (MAE) for the Monte Carlo simulation's weight predictions compared to the actual weights of a Minimum Variance Portfolio (MVP). The x-axis shows the sample size considered.

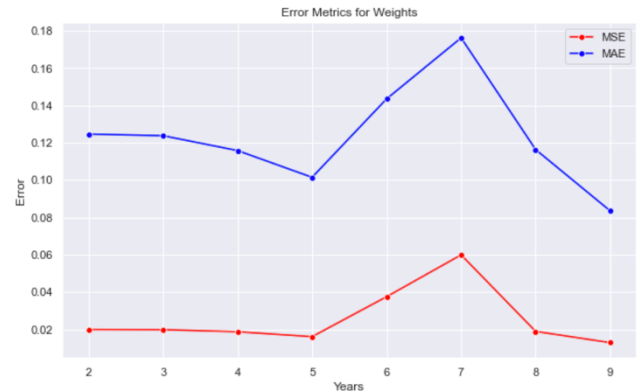
We observe that the MSE (red line) decreases, suggesting a stable prediction accuracy from the simulation model, which decreases as the sample size increases. The MAE (blue line) clearly shows the reduction of errors as the sample size increases.

Error Metrics (Variance) – Exhibit B



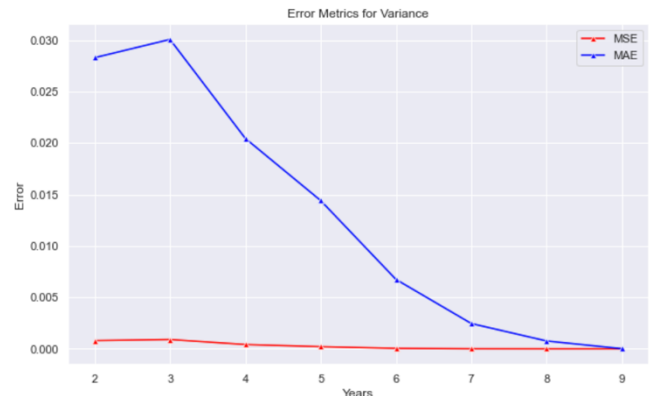
The decreasing trends in both MSE and MAE suggest that the Monte Carlo simulation is producing variance estimates that are progressively closer to the actual portfolio variance as the sample size increases.

Error Metrics (Weights) using Antithetic Variates – Exhibit C



We observe that both the MSE and MAE show a spike, indicating that the precision of the weight estimations varies as the sample size increases.

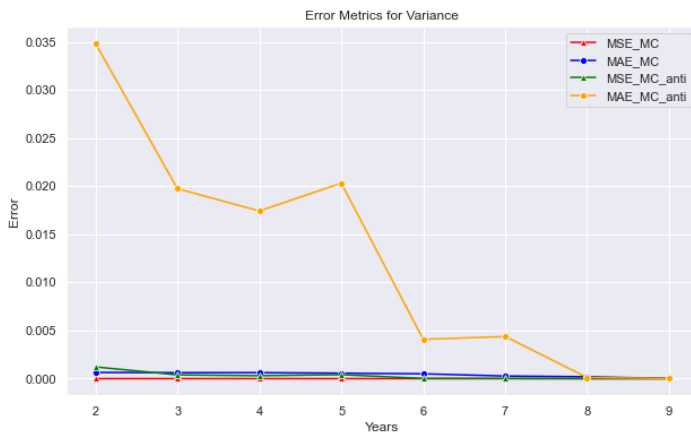
Error Metrics (Variance) using Antithetic Variates – Exhibit D



The declining trends in both the **MSE** and **MAE** suggest that the model's estimations of portfolio variance are becoming more accurate as the sample size increases.

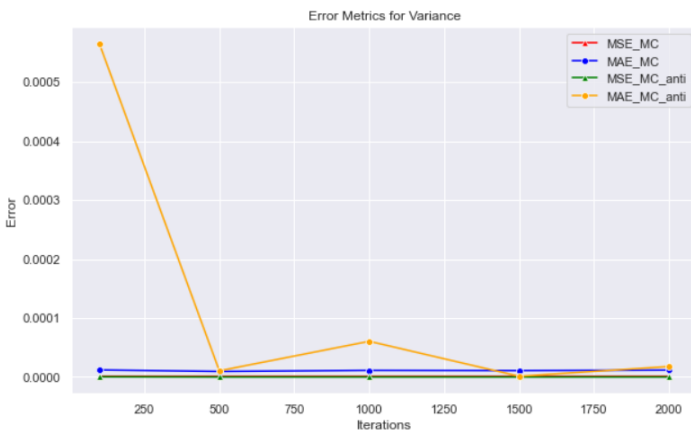
Considering both the error metrics for weights and variance, there's evidence that the Monte Carlo simulation with antithetic variates is refining its predictions as sample size increases.

Comparison of Error Metrics (Variance) Performance b/w Standard and Antithetic Variates Monte Carlo simulations – Exhibit E



Although both error metrics show decreasing trends. We observe that the **MSE** and **MAE** for **MC-anti** show greater error than **MC** for lower sample sizes but as the sample size increases the errors appear to converge.

Performance of Variables (Variance) over Different Simulation Iterations Pertaining to Antithetic Variates – Exhibit F



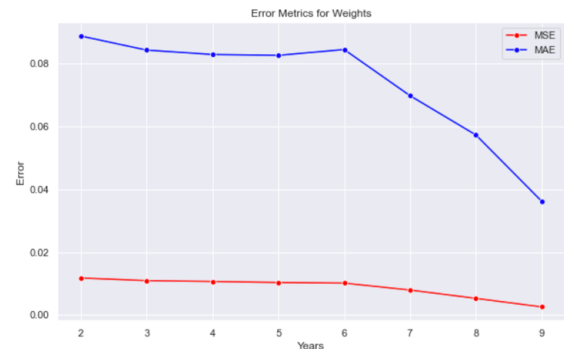
From the above graph, we observe that:

- **MSE_MC (Red Line):** The **MSE** from standard MC simulations appears to be constant as the number of iterations increases. This is expected, as more iterations in Monte Carlo simulations typically lead to a more accurate estimation of variance.

- **MAE_MC (Blue Line):** We observe that the **MAE** from standard MC simulations appears to remain the same throughout.
- **MSE_MC_anti (Green Line):** The **MSE** from MC simulations with antithetic variates shows a similar trend.
- **MAE_MC_anti (Orange Line):** The **MAE** from MC simulations with antithetic variates also decreases with the number of iterations. While we observe a spike in the variance at start, it reduces substantially as we increase the number of iterations. It is able to achieve better results for certain iterations but overall it is unpredictable.

Overall, we see a significant drop in error as iterations increase, especially notable in the **MAE**, which emphasizes the importance of using a sufficient number of iterations in Monte Carlo simulations to achieve accurate results. The convergence of **MSE** and **MAE** for both simulation methods as iterations increase suggests that both the frequency and magnitude of large errors are being reduced.

Error Metrics (Weights) using Moment Matching – Exhibit G



We observe a decreasing trend in weights error metrics for both **MSE** and **MAE** as the sample size increases for **MC-MM**.

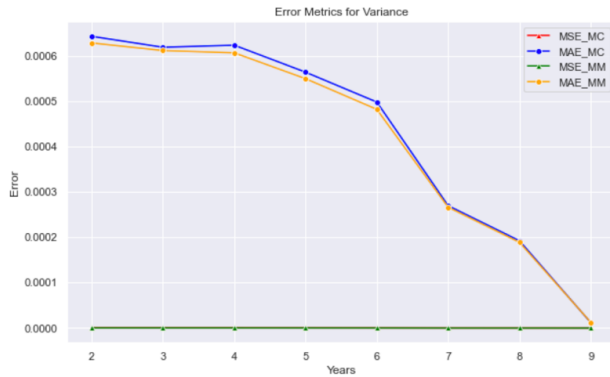
Error Metrics (Variance) using Moment Matching – Exhibit H



We observe that the **MSE** starts at a lower value and shows a slight decrease as the sample size increases. This suggests that the errors in variance estimation squared are getting smaller as the sample size increases. Although **MAE** starts higher than the initial **MSE**, its initial variance value is very low compared to antithetic variance for the same period. Furthermore, it decreases markedly over time, reflecting a

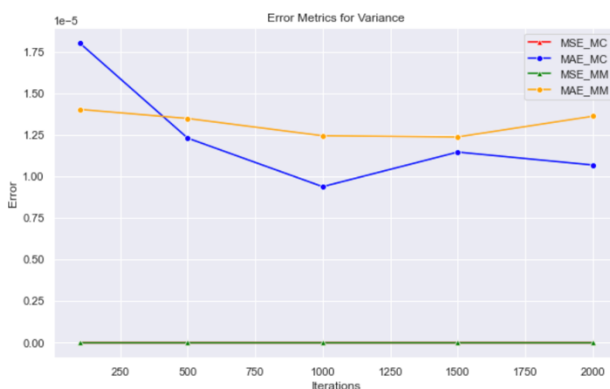
substantial reduction in the average magnitude of the errors in variance estimation.

Comparison of Error Metrics (Variance) b/w Standard and Moment Matching Monte Carlo simulations – Exhibit I



The graph shows that incorporating moment matching into Monte Carlo simulations can improve the accuracy of variance estimations. We can observe that the moment matching technique appears to be effective in reducing both the frequency and magnitude of errors in variance estimation. Furthermore, the substantial decrease in **MAE** is noteworthy, as it implies that the average error is becoming significantly smaller. It's a reminder that while variance reduction techniques can enhance simulation accuracy, they should be periodically reviewed and updated alongside changes in the underlying data or market conditions.

Performance of Variables (Variance) over Different Simulation Iterations Pertaining to Moment Matching -Exhibit J



We observe that the **MSE_MC (Red Line)** almost remains the same as the number of iterations increases, which is typical as more iterations usually lead to a more accurate estimation due to the law of large numbers. **MAE_MC (Blue Line)** also decreases, indicating that the average error in variance estimation reduces as the number of iterations grows. **MSE_MM (Green Line)** is initially lower than standard MC's MSE and remains almost constant as iterations increase. Finally, the **MAE_MM (Orange Line)** starts lower than the **MAE_MC** and closely follows **MAE_MC**.

VIII. RESULTS

The project aimed to improve portfolio optimization by reducing sampling errors, and the introduction of variance reduction techniques in Monte Carlo simulations has yielded significant outcomes. The main findings are as follows:

Our implementation of Monte Carlo simulations with moment matching has demonstrated enhanced precision in the estimation of expected returns. We observed that the expected returns maintained their consistency across different simulation scenarios, indicating that the variance reduction techniques effectively stabilized the prediction model against the sampling errors inherent in historical data-based forecasts.

The Monte Carlo simulations applying antithetic variates exhibited a reduction in variance; however, this technique did not consistently minimize estimation errors across the range of tests. In contrast, moment matching demonstrated superior performance with a noticeable downward trend in error metrics, signifying its robustness and reliability in variance estimation.

These results underscore that certain variance techniques provide better results in refining the predictive capabilities of Monte Carlo simulations depending on the use case and thus optimizing portfolio weights under uncertainty.

Analysis of the weights distribution showed that the application of advanced variance reduction techniques resulted in a dynamic and robust portfolio allocation. Our findings indicate a marked improvement in the allocation strategies, with optimized portfolios showcasing a balanced trade-off between risk and return. This dynamic adjustment ensures that portfolios can better withstand market volatilities and yield consistent performance.

Moment matching has proven to be an indispensable tool in our analysis, facilitating the construction of optimized portfolios that are not only theoretically sound but also empirically validated for practical application.

IX. CONCLUSION

The goal of our project was to explore and validate advanced simulation techniques for reducing sampling errors in portfolio optimization. Our comprehensive analysis revealed that while antithetic variates contribute to variance reduction, the moment matching technique consistently outperformed in terms of reducing the estimation errors for portfolio variance.

Through rigorous testing across a portfolio of 20 stocks, moment matching demonstrated a superior ability to align simulated distributions with their theoretical counterparts, resulting in a more accurate representation of the risk-return profile. The better results in both MSE and MAE for moment matching, particularly for the sample size cases, are a testament to its robustness.

It became evident that the antithetic variates method, despite its merits in variance reduction, did not consistently provide the level of error minimization we aimed for. Its performance, while beneficial in some scenarios, did not match the improvements yielded by moment matching.

In conclusion, this study advances the frontier of quantitative finance by introducing a refined methodological approach that addresses the limitations of traditional optimization techniques. The adoption of variance reduction in portfolio optimization not only elevates the accuracy of simulations but also provides investors with a robust framework for navigating the complexities of modern financial markets.

X. FUTURE TRENDS

- ❖ **Integration with Machine Learning:** As computational power increases, the integration of moment matching with machine learning algorithms presents an avenue for creating more sophisticated models that can adapt to and learn from a continuous influx of market data.
- ❖ **Dynamic Asset Allocation Models:** The increased accuracy in variance estimation paves the way for dynamic asset allocation models that can offer real-time optimization in response to live market changes.
- ❖ **Automated Trading Systems:** There is potential for embedding moment matching into automated trading systems to execute strategies that align closely with the evolving risk-return landscape, enhancing trade execution and portfolio performance.

XI. REFERENCES

1. Jobson, J.D., & Korkie, B. (1980). *Estimation for Markowitz Efficient Portfolios*. *Journal of the American Statistical Association*.
2. Michaud, R.O. (1989). *The Markowitz Optimization Enigma: Is 'Optimized' Optimal?*, *Financial Analysts Journal*.
3. Markowitz, H. (1952). *Portfolio Selection*. *Journal of Finance*.
4. Britten-Jones, M. (1999). *The Sampling Error in Estimates of Mean-Variance Efficient Portfolio Weights*. *Journal of Finance*.
5. Ledoit, O., & Wolf, M. (2004). *A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices*. *Journal of Multivariate Analysis*.
6. Michaud, R.O. (1998). *Efficient Asset Management: A Practical Guide to Stock Portfolio Optimization and Asset Allocation*. Harvard Business School Press.
7. Fabozzi, F.J., Huang, D., & Zhou, G. (2007). *Robust Portfolios: Contributions from Operations Research and Finance*. *Annals of Operations Research*.
8. DeMiguel, V., Garlappi, L., & Uppal, R. (2009). *Optimal Versus Naive Diversification: How Inefficient is the 1/N Portfolio Strategy?*, *Review of Financial Studies*.
9. Chopra, V. K., & Ziemba, W. T. (1993). *The Effect of Errors in Means, Variances, and Covariances on Optimal Portfolio Choice*. *Journal of Portfolio Management*.
10. Fama, E. F., & French, K. R. (1993). *Common Risk Factors in the Returns on Stocks and Bonds*. *Journal of Financial Economics*.

APPENDIX

Code

```

import numpy as np
import pandas as pd
import math
import matplotlib.pyplot as plt
import seaborn as sns; sns.set_theme()

class MeanVariancePortfolio:
    def __init__(self, avg_ret, C, stock_dict):
        self.avg_ret = avg_ret
        self.C = C
        self.stock_dict = stock_dict

    def mean_variance(self):
        """ Function to calculate the minimum variance portfolio using non linear optimization """
        I = np.ones(len(self.avg_ret))
        W = np.linalg.inv(self.C) @ I / (I.T @ np.linalg.inv(self.C) @ I)
        var = W.T @ self.C @ W
        Er = self.avg_ret @ W

        return Er, W, var

    def simulate_efficient_frontier(self):
        """ Function to simulate the efficient frontier """
        return_space = np.linspace(-0.03, 0.05, 1000)
        I = np.ones(len(self.avg_ret))
        A = I.T @ np.linalg.inv(self.C) @ self.avg_ret
        B = self.avg_ret.T @ np.linalg.inv(self.C) @ self.avg_ret
        C = I.T @ np.linalg.inv(self.C) @ I
        D = B * C - A**2

        sigma_space = []
        weights_space = []
        for Er in return_space:
            sigma = np.sqrt(1 / C + (C / D) * (Er - A / C)**2)
            sigma_space.append(sigma)

        return sigma_space, return_space, weights_space

    def simulated_portfolios(self, iterations = 1000):
        return_space = []
        sigma_space = []
        weights_space = []
        for _ in range(0, iterations):
            rn_weights = np.random.uniform(-0.2, 1, len(self.avg_ret))
            weights = (rn_weights / sum(rn_weights)).reshape(-1, 1)
            ret = weights.T @ self.avg_ret
            sig = np.sqrt(weights.T @ self.C @ weights)

```

```

        return_space.append(ret)
        sigma_space.append(sig)
        weights_space.append(weights)

    return sigma_space, return_space

def sample_selection(self, years, samples):
    """ Selects 20 x 12*years from historical data """
    n_stocks = len(self.stock_dict.keys())
    req_samples = (120 - 12*years)*n_stocks
    temp_ret_array = [self.stock_dict[key][:12*years] for key in self.stock_dict]
    temp_ret_array = np.concatenate(temp_ret_array).reshape(n_stocks, 12*years)

    samples = samples[:req_samples]
    samples = np.reshape(samples, (n_stocks,-1))
    ret_array_ny = np.concatenate((temp_ret_array, samples), axis = 1)

    ret_array_ny = np.array(ret_array_ny)
    mean_ny = np.mean(ret_array_ny, axis = 1)
    V_ny = np.cov(ret_array_ny, rowvar = True)  ## We get 20x20 covariance matrix for 8 years of data

    # print("Samples: ", samples.shape)
    # print("Final array: ", ret_array_ny.shape)

    return ret_array_ny, mean_ny, V_ny

def sample_selection_MM(self, ret_array, years, samples):
    n_stocks = len(self.stock_dict.keys())
    req_samples = (120 - 12*years)*n_stocks
    temp_ret_array = [self.stock_dict[key][:12*years] for key in self.stock_dict]
    temp_ret_array = np.concatenate(temp_ret_array).reshape(n_stocks, 12*years)

    samples = samples[:req_samples]
    samples = np.reshape(samples, (n_stocks,-1))
    ret_array_ny = np.concatenate((temp_ret_array, samples), axis = 1)

    std = np.std(ret_array)
    mean = np.mean(ret_array, axis = 1)
    ret_array_ny = np.array(ret_array_ny)
    mean_ny = np.mean(ret_array_ny, axis = 1)
    std_ny = np.std(ret_array_ny)

    samples = ((samples - mean_ny.reshape(-1,1)) * (std / std_ny)) + mean.reshape(-1,1)

    ret_array_samples = np.concatenate((temp_ret_array, samples), axis = 1)
    mean_samples = np.mean(ret_array_samples, axis = 1)
    V_samples = np.cov(ret_array_samples, rowvar = True)

    return ret_array_samples, mean_samples, V_samples

```

```

class VarianceReduction:
    """ Class to implement variance reduction techniques to Monte Carlo simulation
    Parameters:
    iterations: Number of iterations
    samples: Number of samples to be generated """
    def __init__(self, iterations, samples):
        self.u1 = np.array([])
        self.u2 = np.array([])
        self.u1_dict = {i: [] for i in range(2, 10)}
        self.u2_dict = {i: [] for i in range(2, 10)}
        self.iterations = int(iterations)
        self.samples = int(samples/2)

    def standard_normal(self):
        """ Function to generate n samples from a standard normal distribution using Box-Muller method """
        random_numbers = []
        x1 = np.random.rand(self.samples)
        x2 = np.random.rand(self.samples)
        # print("Random x1 numbers generated: ", len(x1))
        # print("Random x2 numbers generated: ", len(x2))
        self.u1 = x1
        self.u2 = x2
        for j in range(self.samples):
            z1 = math.sqrt(-2 * math.log(x1[j])) * math.cos(2 * math.pi * x2[j])
            z2 = math.sqrt(-2 * math.log(x1[j])) * math.sin(2 * math.pi * x2[j])
            random_numbers.append(z1)
            random_numbers.append(z2)

        # print("Standard normal random numbers generated: ", len(random_numbers))
        return random_numbers

    def multivariate_normal(self, mean, cov):
        """ Function to generate n samples from a multivariate normal distribution """
        samples = np.array([])
        x = self.standard_normal()          ## generates 2000 random numbers from standard normal
distribution
        x = np.reshape(x, (-1, len(mean)))
        L = np.linalg.cholesky(cov)
        y = x @ L.T + mean
        for i in y:
            samples = np.append(samples, i)

        # print("Multivariate normal random numbers generated: ", len(samples))
        return samples

    def MC(self, mean, cov, stock_dict):
        Er = {i: [] for i in range(2, 10)}
        W = {i: [] for i in range(2, 10)}

```

```

var = {i: [] for i in range(2, 10)}
for _ in range(self.iterations):
    k = self.multivariate_normal(mean, cov)
    for i in range(2, 10):
        t = int((len(stock_dict.keys())*(120-12*i))/2)
        self.u1_dict[i].append(self.u1[t])
        self.u2_dict[i].append(self.u2[t])
        mean_ny, V_ny = MeanVariancePortfolio(mean, cov, stock_dict).sample_selection(i, k)[1:]
        Er_opt, W_opt, var_opt = MeanVariancePortfolio(mean_ny, V_ny, stock_dict).mean_variance()
        W[i].append(W_opt)
        var[i].append(var_opt)
        Er[i].append(Er_opt)

Er = {i: np.mean(Er[i]) for i in Er}
var = {i: np.mean(var[i]) for i in var}
W = {i: np.mean(W[i], axis = 0) for i in W}

return Er, W, var

def MC_antithetic_variates(self, mean, cov, stock_dict):
    random_numbers = {i: [] for i in range(2, 10)}
    Er_anti = {i: [] for i in range(2, 10)}
    W_anti = {i: [] for i in range(2, 10)}
    var_anti = {i: [] for i in range(2, 10)}
    for j in range(self.iterations):
        for i in range(2, 10):
            t = int((len(stock_dict.keys())*(120-12*i))/4)
            a_u1 = [1 - u for u in self.u1_dict[i][j][:t]]
            a_u1 = np.concatenate((self.u1_dict[i][j][:t], a_u1))
            a_u2 = [1 - u for u in self.u2_dict[i][j][:t]]
            a_u2 = np.concatenate((self.u2_dict[i][j][:t], a_u2))
            # print(self.u1_dict[i][j][:t][:5])
            # print(a_u1[t:t+5])
            # print(len(a_u1))
            z = t*2
            for k in range(z):
                z1 = math.sqrt(-2 * math.log(a_u1[k])) * math.cos(2 * math.pi * a_u2[k])
                z2 = math.sqrt(-2 * math.log(a_u1[k])) * math.cos(2 * math.pi * a_u2[k])
                random_numbers[i].append(z1)
                random_numbers[i].append(z2)

s = random_numbers[i]
# for i in range(2, 10):
mean_ny, V_ny = MeanVariancePortfolio(mean, cov, stock_dict).sample_selection(i, s)[1:]
Er_opt, W_opt, var_opt = MeanVariancePortfolio(mean_ny, V_ny, stock_dict).mean_variance()
W_anti[i].append(W_opt)
var_anti[i].append(var_opt)
Er_anti[i].append(Er_opt)

```

```

Er_anti = {i: np.mean(Er_anti[i]) for i in Er_anti}
var_anti = {i: np.mean(var_anti[i]) for i in var_anti}
W_anti = {i: np.mean(W_anti[i], axis = 0) for i in W_anti}

# print("Random numbers: ", len(random_numbers[2]))
# print("Random numbers: ", len(random_numbers[3]))
# print("Random numbers: ", len(random_numbers[4]))
# print("Random numbers: ", len(random_numbers[5]))

return Er_anti, W_anti, var_anti

def MC_moment_matching(self, ret_array, mean, cov, stock_dict):
    Er = {i: [] for i in range(2, 10)}
    W = {i: [] for i in range(2, 10)}
    var = {i: [] for i in range(2, 10)}
    for _ in range(self.iterations):
        k = self.multivariate_normal(mean, cov)
        for i in range(2, 10):
            # t = int((len(stock_dict.keys())*(120-12*i))/2)
            # self.u1_dict[i].append(self.u1[:t])
            # self.u2_dict[i].append(self.u2[:t])
            mean_ny, V_ny = MeanVariancePortfolio(mean, cov, stock_dict).sample_selection_MM(ret_array, i, k)[1:]
            Er_opt, W_opt, var_opt = MeanVariancePortfolio(mean_ny, V_ny, stock_dict).mean_variance()
            W[i].append(W_opt)
            var[i].append(var_opt)
            Er[i].append(Er_opt)

    Er = {i: np.mean(Er[i]) for i in Er}
    var = {i: np.mean(var[i]) for i in var}
    W = {i: np.mean(W[i], axis = 0) for i in W}

    return Er, W, var

class ErrorMetrics:
    def __init__(self, x, x_hat):
        self.x = x.tolist()
        self.x_hat = x_hat.tolist()

    def MSE(self):
        if isinstance(self.x, float):
            return (self.x - self.x_hat)**2
        else:
            temp = [(self.x[i] - self.x_hat[i])**2 for i in range(len(self.x))]
            return sum(temp) / len(self.x)

    def MAE(self):
        if isinstance(self.x, float):
            return abs(self.x - self.x_hat)
        else:

```

```

        temp = [abs(self.x[i] - self.x_hat[i]) for i in range(len(self.x))]
        return sum(temp) / len(self.x)
stock_data = pd.DataFrame(pd.read_csv('C:/Users/ADITYA/Desktop/Fin Mod/Data/Stock_v.csv'))

stock_data = stock_data.drop(['PERMNO'], axis=1)

key_list = stock_data['TICKER'].unique().tolist()
stock_dict = {key: [] for key in key_list}
print(f"Number of stocks considered: {len(stock_dict.keys())}")
print(f"List of stocks considered: \n {key_list}")

for i in range(len(stock_data)):
    if stock_data['TICKER'][i] in key_list:
        stock_dict[stock_data['TICKER'][i]].append(stock_data['RET'][i])

avg_ret = [np.mean(stock_dict[key]) for key in stock_dict]
avg_ret = np.array(avg_ret)                                ## 1 x 20 matrix of average returns

ret_array = np.array([])
for key in stock_dict:
    ret_array = np.append(ret_array, stock_dict[key])

ret_array = np.reshape(ret_array, (len(key_list), -1))    ## 20 x 120 matrix of returns

C = np.cov(ret_array, rowvar = True)
MVP = MeanVariancePortfolio(avg_ret, C, stock_dict)
ret_mvp, W_mvp, var_mvp = MVP.mean_variance()
sigma_eff, return_eff, weights_eff = MVP.simulate_efficient_frontier()
sigma_sim, return_sim = MVP.simulated_portfolios(10000)

print('Expected Return(MVP):    {:.8f}'.format(ret_mvp))
print('Variance(MVP):          {:.8f}'.format(var_mvp))
print('Weights(MVP):           {}'.format(W_mvp))
plt.figure(figsize=(10, 6))
plt.plot(sigma_eff, return_eff, color='red', linewidth = 1)
plt.scatter(sigma_sim, return_sim, color='blue', s = 3)
plt.scatter(np.sqrt(var_mvp), ret_mvp, color = 'green', s = 15)
plt.legend(['Efficient Frontier', 'Simulated Portfolios', 'Minimum Variance Portfolio'])
plt.xlabel('Standard Deviation')
plt.ylabel('Expected Return')
plt.title('Efficient Frontier')
plt.show()
methods = VarianceReduction(1000, 2400)
Er, W, var = methods.MC(avg_ret, C, stock_dict)
print('Expected Return(MC):    ', Er)
print('Variance(MC):          ', var)
print('Weights(MC):           ', W)
## Calculating error metrics for each key in Er, W and var
MSE_WeightsMC = []

```

```

MAE_WeightsMC = []
for key in W:
    error = ErrorMetrics(W_mv, W[key])
    MSE_WeightsMC.append(error.MSE())
    MAE_WeightsMC.append(error.MAE())

## Plot MSE and MAE for weights for years in range 2 to 8 use seaborn library
plt.figure(figsize=(10, 6))
sns.lineplot(x = list(W.keys()), y = MSE_WeightsMC, color = 'red', label = 'MSE', marker = 'o')
sns.lineplot(x = list(W.keys()), y = MAE_WeightsMC, color = 'blue', label = 'MAE', marker = 'o')
plt.xlabel('Years')
plt.ylabel('Error')
plt.title('Error Metrics for Weights')
plt.legend()
plt.show()

## Calculating error metrics for Var
MSE_varMC = []
MAE_varMC = []
for key in var:
    error = ErrorMetrics(var_mv, var[key])
    MSE_varMC.append(error.MSE())
    MAE_varMC.append(error.MAE())

## Plot MSE and MAE for variance for years in range 2 to 8 use seaborn library
plt.figure(figsize=(10, 6))
sns.lineplot(x = list(var.keys()), y = MSE_varMC, color = 'red', label = 'MSE', marker = '^')
sns.lineplot(x = list(var.keys()), y = MAE_varMC, color = 'blue', label = 'MAE', marker = '^')
plt.xlabel('Years')
plt.ylabel('Error')
plt.title('Error Metrics for Variance')
plt.legend()
plt.show()

Er_anti, W_anti, var_anti = methods.MC_antithetic_variates(avg_ret, C, stock_dict)

print('Expected Return(MC_antithetic_variates): ', Er_anti)
print('Variance(MC_antithetic_variates): ', var_anti)
print('Weights(MC_antithetic_variates): ', W_anti)

## Plot MSE and MAE for weights for years in range 2 to 8 use seaborn library
MSE_WeightsMC_anti = []
MAE_WeightsMC_anti = []
for key in W_anti:
    error = ErrorMetrics(W_mv, W_anti[key])
    MSE_WeightsMC_anti.append(error.MSE())
    MAE_WeightsMC_anti.append(error.MAE())

## Plot MSE and MAE for weights for years in range 2 to 8 use seaborn library
plt.figure(figsize=(10, 6))
sns.lineplot(x = list(W_anti.keys()), y = MSE_WeightsMC_anti, color = 'red', label = 'MSE', marker = 'o')
sns.lineplot(x = list(W_anti.keys()), y = MAE_WeightsMC_anti, color = 'blue', label = 'MAE', marker = 'o')

```

```

plt.xlabel('Years')
plt.ylabel('Error')
plt.title('Error Metrics for Weights')
plt.legend()
plt.show()

## Plot MSE and MAE for variance for years in range 2 to 8 use seaborn library
MSE_varMC_anti = []
MAE_varMC_anti = []
for key in var_anti:
    error = ErrorMetrics(var_mvp, var_anti[key])
    MSE_varMC_anti.append(error.MSE())
    MAE_varMC_anti.append(error.MAE())

## Plot MSE and MAE for variance for years in range 2 to 8 use seaborn library
plt.figure(figsize=(10, 6))
sns.lineplot(x = list(var_anti.keys()), y = MSE_varMC_anti, color = 'red', label = 'MSE', marker = '^')
sns.lineplot(x = list(var_anti.keys()), y = MAE_varMC_anti, color = 'blue', label = 'MAE', marker = '^')
plt.xlabel('Years')
plt.ylabel('Error')
plt.title('Error Metrics for Variance')
plt.legend()
plt.show()

## PLOT comparison of MSE and MAE for variance in MC and MC_antithetic_variates for years
plt.figure(figsize=(10, 6))
sns.lineplot(x = range(2,10), y = MSE_varMC, color = 'red', label = 'MSE_MC', marker = '^')
sns.lineplot(x = range(2,10), y = MAE_varMC, color = 'blue', label = 'MAE_MC', marker = 'o')
sns.lineplot(x = range(2,10), y = MSE_varMC_anti, color = 'green', label = 'MSE_MC_anti', marker = '^')
sns.lineplot(x = range(2,10), y = MAE_varMC_anti, color = 'orange', label = 'MAE_MC_anti', marker = 'o')
plt.xlabel('Years')
plt.ylabel('Error')
plt.title('Error Metrics for Variance')
plt.legend()
plt.show()

## Simulate MC and MC_antithetic_variates for 100, 500, 1000, 1500, 2000 iterations and plot comparison of the error
metrics
iterations = [100, 500, 1000, 1500, 2000]
MSE_varMC_sim = []
MAE_varMC_sim = []
MSE_varMC_anti_sim = []
MAE_varMC_anti_sim = []

for i in iterations:
    methods_sim = VarianceReduction(i, 2400)
    Er, W, var = methods_sim.MC(avg_ret, C, stock_dict)
    Er_anti, W_anti, var_anti = methods_sim.MC_antithetic_variates(avg_ret, C, stock_dict)

    error = ErrorMetrics(var_mvp, var[key])
    MSE_varMC_sim.append(error.MSE())
    MAE_varMC_sim.append(error.MAE())

```



```

    error = ErrorMetrics(var_mvp, var_anti[key])
    MSE_varMC_anti_sim.append(error.MSE())
    MAE_varMC_anti_sim.append(error.MAE())

## Plot MSE and MAE for variance for years in range 2 to 9 use seaborn library
plt.figure(figsize=(10, 6))
sns.lineplot(x = iterations, y = MSE_varMC_sim, color = 'red', label = 'MSE_MC', marker = '^')
sns.lineplot(x = iterations, y = MAE_varMC_sim, color = 'blue', label = 'MAE_MC', marker = 'o')
sns.lineplot(x = iterations, y = MSE_varMC_anti_sim, color = 'green', label = 'MSE_MC_anti', marker = '^')
sns.lineplot(x = iterations, y = MAE_varMC_anti_sim, color = 'orange', label = 'MAE_MC_anti', marker = 'o')
plt.xlabel('Iterations')
plt.ylabel('Error')
plt.title('Error Metrics for Variance')
plt.legend()
plt.show()

mean_MM, W_MM, var_MM = methods.MC_moment_matching(ret_array, avg_ret, C, stock_dict)
print('Expected Return(MC_moment_matching): ', mean_MM)
print('Variance(MC_moment_matching): ', var_MM)
print('Weights(MC_moment_matching): ', W_MM)

## Calculating error metrics for weights for moment matching
MSE_WeightsMC_MM = []
MAE_WeightsMC_MM = []
for key in W_MM:
    error = ErrorMetrics(W_mvp, W_MM[key])
    MSE_WeightsMC_MM.append(error.MSE())
    MAE_WeightsMC_MM.append(error.MAE())

## Plot MSE and MAE for weights for years in range 2 to 8 use seaborn library
plt.figure(figsize=(10, 6))
sns.lineplot(x = list(W_MM.keys()), y = MSE_WeightsMC_MM, color = 'red', label = 'MSE', marker = 'o')
sns.lineplot(x = list(W_MM.keys()), y = MAE_WeightsMC_MM, color = 'blue', label = 'MAE', marker = 'o')
plt.xlabel('Years')
plt.ylabel('Error')
plt.title('Error Metrics for Weights')
plt.legend()
plt.show()

## Calculating error metrics for Var for moment matching
MSE_varMC_MM = []
MAE_varMC_MM = []
for key in var_MM:
    error = ErrorMetrics(var_mvp, var_MM[key])
    MSE_varMC_MM.append(error.MSE())
    MAE_varMC_MM.append(error.MAE())

## Plot MSE and MAE for variance for years in range 2 to 8 use seaborn library
plt.figure(figsize=(10, 6))
sns.lineplot(x = list(var_MM.keys()), y = MSE_varMC_MM, color = 'red', label = 'MSE', marker = '^')
sns.lineplot(x = list(var_MM.keys()), y = MAE_varMC_MM, color = 'blue', label = 'MAE', marker = '^')
plt.xlabel('Years')
plt.ylabel('Error')
plt.title('Error Metrics for Variance')

```

```

plt.legend()
plt.show()

## PLOT comparison of MSE and MAE for variance in MC and MC_moment matching
plt.figure(figsize=(10, 6))
sns.lineplot(x = range(2,10), y = MSE_varMC, color = 'red', label = 'MSE_MC', marker = '^')
sns.lineplot(x = range(2,10), y = MAE_varMC, color = 'blue', label = 'MAE_MC', marker = 'o')
sns.lineplot(x = range(2,10), y = MSE_varMC_MM, color = 'green', label = 'MSE_MM', marker = '^')
sns.lineplot(x = range(2,10), y = MAE_varMC_MM, color = 'orange', label = 'MAE_MM', marker = 'o')
plt.xlabel('Years')
plt.ylabel('Error')
plt.title('Error Metrics for Variance')
plt.legend()
plt.show()

## Simulate MC and MC_moment_matching for 100, 500, 1000, 1500, 2000 iterations and plot comparison of the error
metrics
iterations = [100, 500, 1000, 1500, 2000]
MSE_varMC_sim2 = []
MAE_varMC_sim2 = []
MSE_varMC_sim_MM = []
MAE_varMC_sim_MM = []

for i in iterations:
    methods_sim = VarianceReduction(i, 2400)
    Er, W, var = methods_sim.MC(avg_ret, C, stock_dict)
    mean_MM, W_MM, var_MM = methods_sim.MC_moment_matching(ret_array, avg_ret, C, stock_dict)

    error = ErrorMetrics(var_mvp, var[key])
    MSE_varMC_sim2.append(error.MSE())
    MAE_varMC_sim2.append(error.MAE())

    error = ErrorMetrics(var_mvp, var_MM[key])
    MSE_varMC_sim_MM.append(error.MSE())
    MAE_varMC_sim_MM.append(error.MAE())

## Plot MSE and MAE for variance for years in range 2 to 9 use seaborn library
plt.figure(figsize=(10, 6))
sns.lineplot(x = iterations, y = MSE_varMC_sim2, color = 'red', label = 'MSE_MC', marker = '^')
sns.lineplot(x = iterations, y = MAE_varMC_sim2, color = 'blue', label = 'MAE_MC', marker = 'o')
sns.lineplot(x = iterations, y = MSE_varMC_sim_MM, color = 'green', label = 'MSE_MM', marker = '^')
sns.lineplot(x = iterations, y = MAE_varMC_sim_MM, color = 'orange', label = 'MAE_MM', marker = 'o')
plt.xlabel('Iterations')
plt.ylabel('Error')
plt.title('Error Metrics for Variance')
plt.legend()
plt.show()

```